LTIMindtree — Live Coding Question Bank

Programs asked in the technical interview (and the upgrade coding round), grouped into math, array and string problems — each with the approach to say aloud, edge cases, complexity and a tested solution.

How the live coding part works

Item	Details
When	Usually after the project discussion — 1 or 2 programs, sometimes a follow-up such as “now do it without built-ins”
Where	Shared editor, online compiler, or Notepad on screen-share; sometimes you only explain the logic aloud
Level	Easy to easy-medium — loops, conditions, arrays, strings, basic math. Clean working logic matters more than speed
Language	Your strongest language (C, C++, Java or Python). Solutions here are in Python; the logic is the same in every language

The 5-step method — use it for every problem

1 · Clarify	2 · Explain	3 · Code	4 · Dry run	5 · Improve
Repeat the problem. Ask about input range, duplicates, case, empty input.	Say the approach in 2–3 lines before typing. Brute force first if unsure.	Clear variable names, one function, no unnecessary built-ins.	Trace one normal input and one edge case (0, 1, empty) aloud.	State time and space complexity and one way to optimise.

If you practise only ten

Prime check · Factorial (recursion) · Fibonacci · Reverse a number · Power of 2 with exception handling · Second largest · Remove duplicates · Reverse a string without built-ins · Palindrome · Count vowels — write each once by hand.

Question index

Tags: **MOST ASKED** — appears in many interview reports · **REPORTED** — named in recent LTIMindtree interview experiences · **COMMON** — standard fresher interview program · **PRACTICE** — follow-up level

A · Math & number problems
A1 Check whether a number is prime
A2 All primes up to N / first N primes
A3 Factorial (iterative and recursive)
A4 Fibonacci series
A5 Reverse the digits of a number
A6 Sum of digits & palindrome number
A7 Armstrong number
A8 GCD and LCM of two numbers
A9 Power of 2 with exception handling
A10 Perfect square & decimal to binary

B · Array problems
B1 Second largest element
B2 Largest, smallest and reverse an array
B3 Count even and odd elements
B4 Remove duplicates (keep order)
B5 Frequency of each element
B6 Find the missing number (1 to n)
B7 Pairs with a given sum
B8 Rotate an array by k positions
B9 Bubble sort
B10 Maximum difference pair & move zeros
B11 Matrix: row sums, column sums, largest per row

C · String problems
C1 Reverse a string without built-ins
C2 Palindrome string
C3 Count vowels & consonants / remove vowels
C4 Character frequency & first non-repeating character
C5 Anagram check
C6 Reverse words & count words
C7 Word frequency for a list of words
C8 Replace a substring without replace()
C9 Balanced parentheses
C10 Longest common prefix

D · Pattern printing (reported)
D1 Star pattern that grows and shrinks
D2 Number triangle & pyramid

Section A — Math & number problems

A1 · Check whether a number is prime

MOST ASKED

PROBLEM
Return True if n is prime.

EXAMPLE
17 → True · 15 → False · 1 → False

APPROACH (say this first)
Try divisors only up to √n: if n has a factor larger than √n, it also has one smaller.

EDGE CASES
n < 2 is not prime; 2 is the only even prime.

COMPLEXITY
O(√n) time · O(1) space

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
```

A2 · All primes up to N / first N primes

MOST ASKED

PROBLEM
Print every prime ≤ n, and the first k primes (asked as “first 15 primes”).

EXAMPLE
n = 20 → 2 3 5 7 11 13 17 19

APPROACH (say this first)
Reuse is_prime() in a loop. For a follow-up, mention the Sieve of Eratosthenes (O(n log log n)).

EDGE CASES
n < 2 gives an empty list.

COMPLEXITY
O(n√n) with is_prime()

```
def primes_upto(n):
    return [x for x in range(2, n + 1) if is_prime(x)]

def first_k_primes(k):
    res, x = [], 2
    while len(res) < k:
        if is_prime(x):
            res.append(x)
        x += 1
    return res
```

A3 · Factorial (iterative and recursive)

MOST ASKED

PROBLEM
Return n! for n ≥ 0.

EXAMPLE
5 → 120 · 0 → 1

APPROACH (say this first)
Recursion: base case n ≤ 1 returns 1, otherwise n × fact(n − 1). Be ready to explain the call stack.

EDGE CASES
0! = 1. Negative input → raise an error or return None (say it aloud).

COMPLEXITY
O(n) time · O(n) stack for recursion

```
def fact(n):
    if n <= 1:              # base case
        return 1
    return n * fact(n - 1)

def fact_iter(n):
    res = 1
    for i in range(2, n + 1):
        res *= i
    return res
```

A4 · Fibonacci series

MOST ASKED

PROBLEM
Print the first n Fibonacci numbers; also return the nth term.

EXAMPLE
n = 7 → 0 1 1 2 3 5 8

APPROACH (say this first)
Keep two variables and slide them forward. If asked recursively, mention it is $O(2^n)$ and memoisation fixes it.

EDGE CASES
n = 0 → nothing; n = 1 → 0.

COMPLEXITY
O(n) time · O(1) extra space

```
def fib_series(n):
    res = []
    a, b = 0, 1
    for _ in range(n):
        res.append(a)
        a, b = b, a + b
    return res
```

A5 · Reverse the digits of a number

REPORTED

PROBLEM
Reverse an integer without converting it to a string.

EXAMPLE
1234 → 4321 · 1200 → 21 · −45 → −54

APPROACH (say this first)
Take the last digit with % 10, push it onto rev with rev × 10 + digit, drop it with // 10.

EDGE CASES
Trailing zeros disappear; handle the sign separately.

COMPLEXITY
O(d) where d = number of digits

```
def reverse_num(n):
    sign = -1 if n < 0 else 1
    n, rev = abs(n), 0
    while n > 0:
        rev = rev * 10 + n % 10
        n //= 10
    return sign * rev
```

A6 · Sum of digits & palindrome number

COMMON

PROBLEM
Return the sum of the digits of n; check whether n reads the same backwards.

EXAMPLE
5431 → 13 · 121 → palindrome

APPROACH (say this first)
Same digit-extraction loop as reverse. Palindrome: compare n with reverse_num(n).

EDGE CASES
Negative numbers are not palindromes.

COMPLEXITY
O(d)

```
def digit_sum(n):
    n, s = abs(n), 0
    while n > 0:
        s += n % 10
        n //= 10
    return s

def is_pal_num(n):
    return n >= 0 and n == reverse_num(n)
```

A7 · Armstrong number

COMMON

PROBLEM
Check whether n equals the sum of its digits each raised to the number of digits.

EXAMPLE
153 = $1^3 + 5^3 + 3^3$ → True · 9474 → True

APPROACH (say this first)
Count digits d first, then add each digit to the power d.

EDGE CASES
Every single-digit number is Armstrong.

COMPLEXITY
O(d)

```
def is_armstrong(n):
    d = len(str(n))
    total, t = 0, n
    while t > 0:
        total += (t % 10) ** d
        t //= 10
    return total == n
```

A8 · GCD and LCM of two numbers

COMMON

PROBLEM
Return the GCD and LCM of a and b.

EXAMPLE
12, 18 → GCD 6 · LCM 36

APPROACH (say this first)
Euclid: gcd(a, b) = gcd(b, a % b) until b is 0. Then LCM = a × b ÷ GCD.

EDGE CASES
gcd(a, 0) = a.

COMPLEXITY
O(log min(a, b))

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a * b // gcd(a, b)
```

A9 · Power of 2 with exception handling

REPORTED

PROBLEM
Read a number from the user and say whether it is a power of 2. Handle invalid input.

EXAMPLE
8 → True · 6 → False · “abc” → Invalid input

APPROACH (say this first)
A power of 2 has exactly one set bit, so n & (n − 1) is 0. Wrap int() in try/except for bad input.

EDGE CASES
0 and negatives are not powers of 2; 1 = 2^0 is.

COMPLEXITY
O(1)

```
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def main():
    try:
        n = int(input("Enter a number: "))
        print(is_power_of_two(n))
    except ValueError:
        print("Invalid input: enter an integer")
```

A10 · Perfect square & decimal to binary

PRACTICE

PROBLEM
Check whether n is a perfect square without sqrt(); convert a decimal number to binary without bin().

EXAMPLE
49 → True · 50 → False · 10 → “1010”

APPROACH (say this first)
Square: increase i while i × i < n. Binary: collect n % 2 remainders, then reverse them.

EDGE CASES
0 is a perfect square; binary of 0 is “0”.

COMPLEXITY
O(√n) · O(log n)

```
def is_perfect_square(n):
    i = 0
    while i * i < n:
        i += 1
    return i * i == n

def to_binary(n):
    if n == 0:
        return "0"
    bits = ""
    while n > 0:
        bits = str(n % 2) + bits
        n //= 2
    return bits
```

Section B — Array problems

B1 · Second largest element

MOST ASKED

PROBLEM
Return the second largest distinct value in one pass (no sorting).

EXAMPLE
[12, 35, 1, 10, 34] → 34 · [5, 5, 3] → 3

APPROACH (say this first)
Track first and second. A bigger value pushes first down to second; a value between them updates second.

EDGE CASES
All equal or single element → None. Duplicates of the max must be skipped.

COMPLEXITY
O(n) time · O(1) space

```
def second_largest(arr):
    first = second = None
    for x in arr:
        if first is None or x > first:
            second, first = first, x
        elif x != first and (second is None
                             or x > second):
            second = x
    return second
```

B2 · Largest, smallest and reverse an array

COMMON

PROBLEM
Find the max and min without max()/min(); reverse the array in place.

EXAMPLE
[4, 9, 1, 7] → max 9, min 1 · reversed [7, 1, 9, 4]

APPROACH (say this first)
One loop for max/min. Reverse with two pointers swapping from both ends towards the middle.

EDGE CASES
Empty array: say you would raise an error.

COMPLEXITY
O(n) time · O(1) space

```
def max_min(arr):
    mx = mn = arr[0]
    for x in arr[1:]:
        if x > mx: mx = x
        if x < mn: mn = x
    return mx, mn

def reverse_in_place(arr):
    i, j = 0, len(arr) - 1
    while i < j:
        arr[i], arr[j] = arr[j], arr[i]
        i += 1; j -= 1
    return arr
```

B3 · Count even and odd elements

REPORTED

PROBLEM
Count how many elements are even and how many are odd.

EXAMPLE
[1, 2, 3, 4, 6] → even 3, odd 2

APPROACH (say this first)
Check x % 2 == 0 for each element (or x & 1 == 0 as a bitwise follow-up).

EDGE CASES
Negative numbers work with % 2 in Python; in C/Java use x % 2 != 0 for odd.

COMPLEXITY
O(n)

```
def count_even_odd(arr):
    even = odd = 0
    for x in arr:
        if x % 2 == 0:
            even += 1
        else:
            odd += 1
    return even, odd
```

B4 · Remove duplicates (keep order)

REPORTED

PROBLEM
Return the array with duplicates removed, keeping the first occurrence order.

EXAMPLE
[3, 1, 3, 2, 1] → [3, 1, 2]

APPROACH (say this first)
Keep a set of values already seen; append a value only the first time. Follow-up: for a sorted array, use two pointers in place.

EDGE CASES
Using set(arr) alone loses the order — mention why you avoided it.

COMPLEXITY
O(n) time · O(n) space

```
def remove_duplicates(arr):
    seen, res = set(), []
    for x in arr:
        if x not in seen:
            seen.add(x)
            res.append(x)
    return res
```

B5 · Frequency of each element

COMMON

PROBLEM
Print how many times each element occurs, in order of first appearance.

EXAMPLE
[2, 3, 2, 5, 3, 2] → 2:3, 3:2, 5:1

APPROACH (say this first)
Use a dictionary (HashMap in Java): count[x] = count.get(x, 0) + 1.

EDGE CASES
Python 3.7+ dictionaries keep insertion order.

COMPLEXITY
O(n) time · O(n) space

```
def frequency(arr):
    count = {}
    for x in arr:
        count[x] = count.get(x, 0) + 1
    return count
```

B6 · Find the missing number (1 to n)

COMMON

PROBLEM
An array holds n − 1 distinct numbers from 1 to n. Find the missing one.

EXAMPLE
[1, 2, 4, 5] (n = 5) → 3

APPROACH (say this first)
Expected sum n(n + 1)/2 minus the actual sum. Mention XOR as an overflow-safe follow-up.

EDGE CASES
Missing number may be 1 or n.

COMPLEXITY
O(n) time · O(1) space

```
def missing_number(arr):
    n = len(arr) + 1
    return n * (n + 1) // 2 - sum(arr)
```

B7 · Pairs with a given sum

COMMON

PROBLEM
Print all pairs (a, b) in the array with a + b = target.

EXAMPLE
[1, 5, 7, -1, 5], target 6 → (1, 5) (1, 5) (7, −1)

APPROACH (say this first)
Brute force with two loops is fine to start ($O(n^2)$); then improve with a dictionary of counts seen so far (O(n)).

EDGE CASES
Clarify with the interviewer whether repeated values give repeated pairs.

COMPLEXITY
$O(n^2)$ → O(n) with hashing

```
def pairs_with_sum(arr, target):
    pairs = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target:
                pairs.append((arr[i], arr[j]))
    return pairs
```

B8 · Rotate an array by k positions

PRACTICE

PROBLEM
Rotate the array to the right by k steps.

EXAMPLE
[1, 2, 3, 4, 5], k = 2 → [4, 5, 1, 2, 3]

APPROACH (say this first)
Use k % n first. Slicing works; the in-place follow-up is reverse all, reverse first k, reverse the rest.

EDGE CASES
k larger than n; k = 0; empty array.

COMPLEXITY
O(n)

```
def rotate_right(arr, k):
    n = len(arr)
    if n == 0:
        return arr
    k %= n
    return arr[n - k:] + arr[:n - k]
```

B9 · Bubble sort

COMMON

PROBLEM
Sort an array in ascending order without sort().

EXAMPLE
[5, 1, 4, 2] → [1, 2, 4, 5]

APPROACH (say this first)
Swap adjacent pairs that are out of order; each pass bubbles the largest to the end. Stop early if no swap happens.

EDGE CASES
Already sorted → one pass with the early-exit flag.

COMPLEXITY
O(n²) worst · O(n) best

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

B10 · Maximum difference pair & move zeros

PRACTICE

PROBLEM
Max of arr[j] − arr[i] with j > i; and move all zeros to the end keeping order.

EXAMPLE
[2, 3, 10, 6, 4, 8, 1] → 8 · [0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]

APPROACH (say this first)
Max difference: track the smallest value seen so far. Move zeros: copy non-zeros forward, fill the rest with 0.

EDGE CASES
Max difference needs at least 2 elements.

COMPLEXITY
O(n) each

```
def max_difference(arr):
    best, low = arr[1] - arr[0], arr[0]
    for x in arr[1:]:
        best = max(best, x - low)
        low = min(low, x)
    return best

def move_zeros(arr):
    k = 0
    for x in arr:
        if x != 0:
            arr[k] = x; k += 1
    for i in range(k, len(arr)):
        arr[i] = 0
    return arr
```

B11 · Matrix: row sums, column sums, largest per row

PRACTICE

PROBLEM
For a 2D matrix, print each row sum, each column sum and the largest value in each row.

EXAMPLE
[[1, 2], [3, 4]] → rows [3, 7] · cols [4, 6] · row max [2, 4]

APPROACH (say this first)
Nested loops: outer over rows, inner over columns. Column sums use matrix[r][c] with c in the outer loop.

EDGE CASES
Assume a rectangular matrix; mention it.

COMPLEXITY
O(rows × cols)

```
def matrix_stats(m):
    rows = [sum(r) for r in m]
    cols = [sum(m[r][c] for r in range(len(m)))
            for c in range(len(m[0]))]
    row_max = [max(r) for r in m]
    return rows, cols, row_max
```

Section C — String problems

C1 · Reverse a string without built-ins

MOST ASKED

PROBLEM
Reverse a string without slicing [::-1] or reversed().

EXAMPLE
“hello” → “olleh”

APPROACH (say this first)
Build the result by putting each character in front, or swap characters from both ends of a list.

EDGE CASES
Empty string; single character.

COMPLEXITY
O(n) with a list (string += is $O(n^2)$ in the worst case — mention it)

```
def reverse_str(s):
    chars = list(s)
    i, j = 0, len(chars) - 1
    while i < j:
        chars[i], chars[j] = chars[j], chars[i]
        i += 1; j -= 1
    return "".join(chars)
```

C2 · Palindrome string

MOST ASKED

PROBLEM
Check whether a string reads the same both ways, ignoring case and non-letters.

EXAMPLE
“Madam” → True · “A man, a plan, a canal: Panama” → True

APPROACH (say this first)
Two pointers from both ends; skip characters that are not letters or digits; compare in lower case.

EDGE CASES
Ask whether case and spaces should be ignored before you code.

COMPLEXITY
O(n) time · O(1) space

```
def is_palindrome(s):
    i, j = 0, len(s) - 1
    while i < j:
        if not s[i].isalnum():
            i += 1
        elif not s[j].isalnum():
            j -= 1
        elif s[i].lower() != s[j].lower():
            return False
        else:
            i += 1; j -= 1
    return True
```

C3 · Count vowels & consonants / remove vowels

REPORTED

PROBLEM
Count vowels and consonants; return the string with vowels removed.

EXAMPLE
“Hello World” → vowels 3, consonants 7 · “Hll Wrld”

APPROACH (say this first)
Check membership in a vowel set; only letters count as consonants (spaces and digits are skipped).

EDGE CASES
Upper-case vowels.

COMPLEXITY
O(n)

```
VOWELS = set("aeiouAEIOU")

def vowel_consonant(s):
    v = c = 0
    for ch in s:
        if ch.isalpha():
            if ch in VOWELS: v += 1
            else: c += 1
    return v, c

def remove_vowels(s):
    return "".join(ch for ch in s
                   if ch not in VOWELS)
```

C4 · Character frequency & first non-repeating character

COMMON

PROBLEM
Count each character; return the first character that appears only once.

EXAMPLE
“swiss” → s:3, w:1, i:1 · first non-repeating = “w”

APPROACH (say this first)
Pass 1 builds the count dictionary; pass 2 walks the string again and returns the first char with count 1.

EDGE CASES
No unique character → return None (or “-1” — ask).

COMPLEXITY
O(n) time · O(k) space

```
def char_freq(s):
    count = {}
    for ch in s:
        count[ch] = count.get(ch, 0) + 1
    return count

def first_unique(s):
    count = char_freq(s)
    for ch in s:
        if count[ch] == 1:
            return ch
    return None
```

C5 · Anagram check

COMMON

PROBLEM
Check whether two strings contain the same characters with the same counts.

EXAMPLE
“listen”, “silent” → True · “rat”, “car” → False

APPROACH (say this first)
Compare character counts (O(n)), or compare sorted(a) == sorted(b) as the simple version (O(n log n)).

EDGE CASES
Different lengths → False immediately. Ask about case and spaces.

COMPLEXITY
O(n)

```
def is_anagram(a, b):
    if len(a) != len(b):
        return False
    return char_freq(a) == char_freq(b)
```

C6 · Reverse words & count words

COMMON

PROBLEM
Reverse the order of words; count the words in a sentence.

EXAMPLE
“I love coding” → “coding love I” · 3 words

APPROACH (say this first)
split() handles multiple spaces; reverse the list with two pointers if built-ins are not allowed.

EDGE CASES
Leading, trailing and repeated spaces.

COMPLEXITY
O(n)

```
def reverse_words(s):
    words = s.split()
    i, j = 0, len(words) - 1
    while i < j:
        words[i], words[j] = words[j], words[i]
        i += 1; j -= 1
    return " ".join(words)

def count_words(s):
    return len(s.split())
```

C7 · Word frequency for a list of words

REPORTED

PROBLEM
Given sentence S and a list of words, return how many times each word appears in S.

EXAMPLE
S = “apple banana apple orange banana apple grape banana”, words = [apple, banana, orange, grape] → [3, 3, 1, 1]

APPROACH (say this first)
Count every word of S once in a dictionary, then look up each query word (missing → 0).

EDGE CASES
Query word absent from S gives 0.

COMPLEXITY
O(len(S) + len(words))

```
def word_counts(S, words):
    count = {}
    for w in S.split():
        count[w] = count.get(w, 0) + 1
    return [count.get(w, 0) for w in words]
```

C8 · Replace a substring without replace()

REPORTED

PROBLEM
Replace every occurrence of old with new in s.

EXAMPLE
s = “I like cats, cats are cute”, old = “cats”, new = “dogs” → “I like dogs, dogs are cute”

APPROACH (say this first)
Scan with index i; if s[i : i + len(old)] matches, append new and jump ahead; else append s[i].

EDGE CASES
old not found → string unchanged; old must not be empty.

COMPLEXITY
O(n × m)

```
def replace_sub(s, old, new):
    res, i, m = [], 0, len(old)
    while i < len(s):
        if s[i:i + m] == old:
            res.append(new)
            i += m
        else:
            res.append(s[i])
            i += 1
    return "".join(res)
```

C9 · Balanced parentheses

PRACTICE

PROBLEM
Check whether every bracket ( ) [ ] { } is closed in the right order.

EXAMPLE
“{[()]}” → True · “([)]” → False · “((” → False

APPROACH (say this first)
Push opening brackets on a stack; on a closing bracket the top must be its partner. Stack must be empty at the end.

EDGE CASES
Closing bracket with an empty stack → False.

COMPLEXITY
O(n) time · O(n) space

```
def is_balanced(s):
    pair = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pair:
            if not stack or stack.pop() != pair[ch]:
                return False
    return not stack
```

C10 · Longest common prefix

PRACTICE

PROBLEM
Return the longest prefix shared by all strings in a list.

EXAMPLE
[“flower”, “flow”, “flight”] → “fl” · [“dog”, “car”] → “”

APPROACH (say this first)
Start with the first word as the prefix; shorten it until every other word starts with it.

EDGE CASES
Empty list → “”.

COMPLEXITY
O(total characters)

```
def longest_common_prefix(words):
    if not words:
        return ""
    prefix = words[0]
    for w in words[1:]:
        while not w.startswith(prefix):
            prefix = prefix[:-1]
    return prefix
```

Section D — Pattern printing (reported)

D1 · Star pattern that grows and shrinks

REPORTED

PROBLEM
Print stars from 1 up to n and back down to 1.

EXAMPLE
n = 3 → * / ** / *** / ** / *

APPROACH (say this first)
Loop 1..n printing i stars, then n − 1 down to 1. Explain the two loops before writing.

EDGE CASES
n = 1 prints a single star.

COMPLEXITY
O(n^2) characters printed

```
def grow_shrink(n):
    lines = []
    for i in range(1, n + 1):
        lines.append("*" * i)
    for i in range(n - 1, 0, -1):
        lines.append("*" * i)
    return lines
```

D2 · Number triangle & pyramid

REPORTED

PROBLEM
Print a right-angled number triangle and a centred star pyramid.

EXAMPLE
n = 3 → 1 / 1 2 / 1 2 3 · pyramid rows “ *”, “ ***”, “*****”

APPROACH (say this first)
Row i prints 1..i. Pyramid: (n − i) spaces then (2i − 1) stars.

EDGE CASES
Ask whether a space is needed between numbers.

COMPLEXITY
O(n²)

```
def number_triangle(n):
    return [" ".join(str(j) for j in range(1, i + 1))
            for i in range(1, n + 1)]

def pyramid(n):
    return [" " * (n - i) + "*" * (2 * i - 1)
            for i in range(1, n + 1)]
```

Java & C quick notes

Need	Java	C
Read input	Scanner sc = new Scanner(System.in); int n = sc.nextInt();	scanf("%d", &n);
String length / char	s.length(), s.charAt(i)	strlen(s), s[i] (needs <string.h>)
Build a string	StringBuilder sb; sb.append(c); sb.reverse()	char array + '\0' terminator
Frequency count	HashMap<Character, Integer> and getOrDefault()	int count[256] = {0}; count[s[i]]++;
Bad input	catch InputMismatchException / NumberFormatException	check the return value of scanf()